

Assignment 4: Large Scale Machine Learning

Course: Data Engineering-II

Introduction

In this assignment, you will explore the challenges and solutions involved in scaling machine learning pipelines to distributed settings. You will cover distributed hyperparameter tuning, deep learning parallelism, and privacy-preserving federated learning.

Points and Grading

- **Total Points:** 2.0.
 - **Task 1 (Mandatory):** 1.0 point.
 - **Task 2 (For extra credit):** 1.0 point.
 - **Task 3 (Optional):** (0 points).

Submission: For all tasks, include screenshots in the report to show completeness, report experimental results, and answer questions in a well-formatted manner.

Task 1: Distributed Machine Learning (Mandatory, Point=1)

1.1: Hyperparameter Tuning

A common task in all machine learning pipelines is to tune so-called *hyperparameters* of the ML algorithms. These are parameters involved in the training process that are not learned from the data itself, but typically related to the optimization routine used. Examples include the learning rate for SGD, penalty terms, batch sizes, and many other things. A good introduction to the problem with examples for scikit-learn using cross-validation is found here: [3.2. Tuning the hyperparameters of an estimator — scikit-learn 1.6.1 documentation](#).

The problem is that since the ML model training is often expensive, hyperparameter tuning becomes a highly computationally demanding and hence time-consuming process. Fortunately, it is possible to treat it as a task-parallel problem, where the ML-model training is systematically repeated for different combinations of hyperparameters, in the search for optimally performing combinations in the global space of hyperparameters. Strategies range from simple ones like exhaustive Random search and Grid Search, to more sophisticated alternatives like Bayesian Optimization. There are several established frameworks for distributed parameter tuning such as [RayTune](#), [HyperOpt](#) and [DaskML](#). Spark, which you encountered in Data Engineering I, also provides [functionality for models made in Spark MLlib](#).

Your task now is to use scikit-learn to train a RandomForestClassifier ([RandomForestClassifier — scikit-learn 1.6.1 documentation](#)) for predicting the forest cover type. The dataset can be downloaded directly via the sklearn API ([fetch_covtype — scikit-learn 1.6.1 documentation](#)) and

there are several notebooks on Kaggle where you can learn about the data and background to the dataset ([Forest Cover Type Prediction | Kaggle](#)).

Do the following:

1. Set up the problem using scikit-learn. Explore the parameters used in the Random Forest implementation. Hint: use the `get_param()` method. Train the model using the default parameters.
2. Implement a distributed hyperparameter tuning pipeline for tuning the parameters `max_depth`, `n_estimators` and `ccp_alpha` using Ray Tune and an exhaustive Grid Search strategy. Configure, deploy and run it on up to 3 VMs of “small” flavor.

Please report on:

1. The hyperparameters found and the associated cross-validation score. How does it compare to the score for the default parameters?
2. The time taken to complete the tuning, when using 1, 2 and 3 VMs of “small” flavor.

1.2 Distributed training of deep neural networks

Another challenge that calls for large scale computing is when training very large models, and/or when the training dataset is very large. Again, distributed computing provides a solution to speed up the training process, but now it is less straight-forward how to achieve parallelism.

In this task, which will be mostly theoretical in nature (unfortunately we are not able to provide access to GPU clusters), you will explore available approaches to scale training by leveraging multiple GPUs/TPUs.

PyTorch

Since you are familiar with the PyTorch library, start by reading up on this background:

- Background on data parallelism: [Data parallelism - Wikipedia](#)
- Implementation of DataParallel training in PyTorch: [PyTorch Distributed: Experiences on Accelerating Data Parallel Training](#)
- Documentation for PyTorch Distributed: [PyTorch Distributed Overview — PyTorch Tutorials 2.7.0+cu126 documentation](#)

Keras/TensorFlow

Then study and do a tutorial using another broadly used ML stack - Keras/Tensorflow. While the APIs and implementations are different, you will recognize the common themes for parallelism: Distributed Tensorflow (on CPU): [Distributed training with TensorFlow](#). Then do this tutorial in Google Colab: [Distributed training with TensorFlow](#)

Equipped with this background knowledge, explain (in max 0.5 page 11pt Arial) the main difference between data parallel training strategies and model parallel training strategies. In which situations are the respective strategies most appropriate to use? Assuming you also need to do hyperparameter tuning, in which situations would you consider distributing the training of individual neural networks rather than distributing test cases for the tuning pipeline?

Task 2: Federated Learning (Extra, Point=1)

Federated Learning (FL) allows multiple organizations to jointly train a model without disclosing their private data.

Task: Use **PySyft** to simulate a privacy-preserving federated environment.

1. **Simulation:** Set up two virtual workers (e.g., "Client A" and "Client B") and distribute the **MNIST** dataset between them.
2. **Training:** Perform federated training where the model is sent to workers for local updates and then aggregated.
3. **Analysis:**
 - Explain how PySyft's "PointerTensor" prevents the data scientist from accessing raw private data.
 - Compare the accuracy of the federated model against a model trained only on a single worker's local data.

Written Reflection:

- Write a reflection (max 0.5 page) on how FL differs from distributed ML regarding statistical and system heterogeneity.
- Discuss how FL relates to the **Parameter Server** strategy.

Note:

- PySyft version: APIs differ between 0.7 (`VirtualWorker`/`PointerTensor`) and 0.8+ (`Dataset`/`ActionObject`). Keep this in mind while doing the analysis.
- If you are using older version, make sure the python environment supports it. Example: syft version 0.2.9 is supported by python 3.7.
- If you are using the latest version of pysyft (recommended):
 - Use pip install "syft>=0.8.0"
 - Send "model.state_dict()", not the model object itself. Otherwise your error might look like "Error when creating action object ...".
 - Add serialization rules: Return "dict" (not tuples); detach gradients via "loss.detach().item()" before returning.
 - Mention "dev_mode=True": Required to bypass manual UI approval for simulation workflows
 - Document "@syft_function": Remote code must use this decorator with "input_policy"/ "output_policy" in 0.8+. Model class and imports must be re-defined inside "@syft_function"
- Ensure you partition the MNIST dataset so that each node trains on a unique, non-overlapping subset.

Task 3: Infrastructure & Scalability (Optional, Point=0)

This task is non-mandatory and focuses on the overhead of distributed systems.

Task: Profiling Communication vs. Computation

1. **Experiment:** Run a large-scale hyperparameter sweep or training job on a single "large" VM versus a cluster of 4 "small" VMs using Ray.
2. **Monitoring:** Analyze **Network I/O** and **CPU Utilization** across the nodes.
3. **Discussion:**
 - Did the cluster outperform the single large node? Explain your findings considering network latency and data serialization.
 - Briefly research the role of **Kubernetes (K8s)** and **KubeFlow**. How do these tools help manage the distributed ML pipelines explored in Task 1?